

Spike-based Belief Propagation in Nonlinear Dynamical Systems

Sepideh Adamiat¹[0009–0009–7269–1596], Hongye Wang², Wouter M. Kouw¹[0000–0002–0547–4817], and Bert de Vries¹[0000–0003–0839–174X]

¹ Electrical Engineering Department, Eindhoven University of Technology, Eindhoven, the Netherlands

² Biomedical Signals and Systems, Faculty of Electrical Engineering, Mathematics and Computer Science (EEMCS), University of Twente, Enschede, The Netherlands

Abstract. This paper presents a Bayesian control framework that integrates spike-based dynamics with probabilistic inference for adaptive control. Bayesian inference is widely regarded as a core computational principle of brain function, providing a normative framework for perception, decision-making, and learning under uncertainty. By combining a biologically inspired spiking neural model with Bayesian inference principles, we propose a brain-like control algorithm capable of operating in uncertain environments. We use the mountain car parking problem as a benchmark with non-linear dynamics. Our results demonstrate that the proposed controller can successfully update states in real time and generate goal-directed action plans through spike-driven dynamics. The results highlight the proposed model’s potential as a bridge between computational neuroscience and probabilistic control theory.

Keywords: Bayesian control · Belief propagation · Brain-inspired computing · Neuromorphic computing · Spiking neural networks · State estimation

1 Introduction

Modeling the human brain remains a central challenge in modern science, motivated in part by the remarkable efficiency of biological intelligence. Humans learn and adapt using far less energy and data than contemporary artificial systems; for instance, the brain operates on roughly 20 Watts [1], whereas modern AI systems typically require many orders of magnitude more computational resources.

Computational neuroscience has evolved from detailed biophysical simulations to theoretical frameworks explaining higher-level cognition. These approaches are broadly categorized as top-down and bottom-up. Top-down methods, such as the Bayesian brain hypothesis [2], and the free energy principle [3], provide normative accounts of perception and decision-making, where normative refers to specifying how an ideal agent should optimally infer and act under uncertainty. However, such approaches often abstract away biological constraints. Bottom-up approaches, including spiking neural networks (SNNs) [4] emphasize neural

dynamics and biophysical realism. SNNs hold promise for improved energy efficiency due to their event-driven computation, where processing occurs only when spikes are generated, enabling sparse communication and reduced power consumption, particularly on neuromorphic hardware [5].

Recent work aims to bridge these perspectives by developing brain-inspired algorithms that combine principled inference with neural implementation [6–9]. In this paper, we follow this direction by integrating Bayesian inference with spiking neural networks.

To enable scalable inference, we adopt a distributed message-passing formulation on factor graphs [10], also known as Belief Propagation (BP). This approach decomposes global inference into local computations, aligning with the parallel and decentralized organization of neural systems. BP has been applied to nonlinear control and multi-agent systems [11–13] and is well-suited to event-driven (reactive) implementations [14].

Several studies have explored BP in SNNs [8]. A close relationship between BP and neurodynamic models was established in [15]. However, that work was limited to Markov random fields, which constitute a subset of factor graphs. In addition, the considered formulation focused on binary-valued variables. Steimer et al. [7] extended this approach by implementing BP in SNNs using a network of interconnected liquid state machines [16]. Nevertheless, these approaches primarily focused on binary-valued variables.

Many real-world inference and control problems involve continuous-valued states, such as positions and velocities, requiring Gaussian belief representations. A spike-based framework for Gaussian BP was introduced in [17], demonstrating belief updating through linear factor nodes and implementing two fundamental Bayesian inference problems: Kalman filtering and linear regression. However, the framework was limited to linear operations and did not support multivariate belief updates.

The main contributions of this paper are:

1. Integration of a spike-based node function into an event-driven BP framework, capable of performing state estimation and action planning in real time.
2. A spike-based message passing implementation of multivariate and nonlinear belief updates for the Mountain Car problem [18].

The proposed framework successfully solves the Mountain Car problem via message passing-based inference on a factor graph and provides a concrete step toward bridging the algorithmic level of probabilistic inference on factor graphs and their implementation via spiking neural dynamics.

2 Problem Statement

The Mountain Car problem is a classical benchmark in control theory and reinforcement learning. This environment consists of a car located in a valley between two hills, intending to reach a goal position. A schematic representation of the

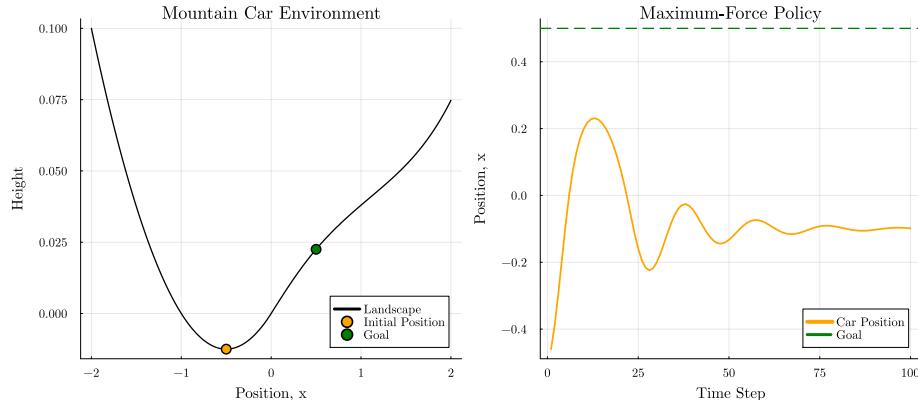


Fig. 1: (left) The Mountain Car environment; (right) trajectory of the car position when the agent naively applies the maximum forward engine force at every time step. The orange curve shows the resulting position over time, while the dashed green line indicates the goal position. This strategy fails to reach the goal state; see Section 2.

environment is shown in Fig. 1 (left), illustrating the mountain profile and the car’s position. In this paper, we use the formulation introduced in [19].

In this problem, the vehicle is assumed to have a limited engine force. The car’s trajectory when maximum force is continuously applied toward the goal is illustrated in Fig. 1 (right). As observed, the car cannot reach the goal position due to insufficient power.

To successfully reach the goal, the control policy must first drive the car in the opposite direction (toward the left hill) to build up sufficient momentum. The accumulated kinetic energy can then propel the car toward the right hill, ultimately reaching the goal position.

To solve this problem, we follow the probabilistic framework proposed in [11]. We consider an agent with an internal state defined as $s_t = [x_t, \dot{x}_t]$, where x_t and $\dot{x}_t$ denote the position and velocity of the agent at time step t , respectively. At each time step, the agent applies an action u_t to the environment and receives an observation y_t . The agent maintains probabilistic beliefs over its internal states, observations, and actions, all of which are modeled as Gaussian random variables.

To describe the relationship among hidden states, observations, and actions, we employ a generative model that provides a probabilistic description of how observations are generated. In this work, the probabilistic dependencies among the variables in the generative model are represented graphically using factor graphs. A biological agent continuously exchanges information with its environment and updates the internal states of its model to minimize expected surprise about sensory observations [20].

Following this framework, we design a synthetic agent that continuously updates its beliefs about hidden states and control variables based on incoming observations, thereby selecting actions that increase the likelihood of achieving desired observations. These belief updates are performed using BP [21] on the factor graph representation of the generative model.

The primary objective of this work is to bridge the gap between BP on factor graphs and the spike-based dynamics observed in biological neural systems. The distributed and local computations performed by BP algorithms have motivated their study as plausible models of neural information processing and probabilistic inference [21, 22]. In particular, our main contribution is a spike-based implementation of the nonlinear and multivariate state-transition factor, $p(s_t | s_{t-1}, u_t)$, that operates on Gaussian beliefs. This factor reflects the Mountain Car environment’s physical dynamics and is one of the most challenging components of the BP procedure.

Accordingly, the main problems addressed in this work are:

1. a spike-based representation of belief parameters (Section 3.5).
2. a spiking neural implementation of a nonlinear and multivariate state-transition factor (Section 4.2).
3. the integration of this neural factor into a BP-based inference framework (Section 4.4).

3 Technical Background

This section presents the technical background required for the proposed model. Subsections 3.1, 3.2, 3.3 describe the probabilistic representation, the inference algorithm, and the control algorithm, respectively. Subsections 3.4, 3.5, 3.6 then introduce the neural dynamics, the encoding and decoding of beliefs in the spike domain, and the methods used to implement the desired functions using networks of spiking neurons.

3.1 Model Representation by Factor Graphs

To enable efficient and scalable Bayesian inference, we adopt a factor graph representation, which expresses a factorized probabilistic model and supports inference via local message (belief) passing. This structure promotes modularity and aligns naturally with neural computation, where local interactions correspond to message exchanges.

We use Forney-style Factor Graphs (FFGs), in which edges represent variables and nodes represent factors [23]. Since each edge connects at most two nodes, branching (equality) nodes are introduced to model variables that appear in more than two factors [24], thus enabling a graphical representation for any factorized probabilistic model.

Consider a state space model,

$$p(s_t, y_t, u_t, s_{t-1}) = \underbrace{p(y_t | s_t)}_{\text{observation}} \underbrace{p(s_t | s_{t-1}, u_t)}_{\text{state transition}} \underbrace{p(u_t)p(s_{t-1})}_{\text{priors}}, \quad (1)$$

which describes the relationship between successive states s_{t-1} and s_t , outputs y_t and control variables (actions) u_t . At time step t , after selecting an action $u_t = \hat{u}_t$ and having observed $y_t = \hat{y}_t$, the FFG representation of this model is shown in Fig. 2.

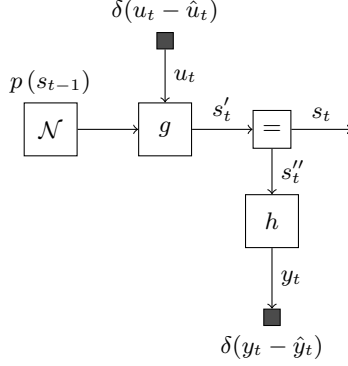


Fig. 2: An FFG representation of one time step of the state space model defined in (1), constrained by $u_t = \hat{u}_t$ and $y_t = \hat{y}_t$.

This FFG comprises four types of nodes. A data node (small black box) represents a delta constraint on a variable with a known value. For example, an observed variable y_t with measured value $\hat{y}_t$ is represented by the factor $\delta(y_t - \hat{y}_t)$. A Gaussian factor node encodes a Gaussian distribution parameterized by its mean and covariance. An equality (branch) node enforces consistency among connected variables and enables Bayesian belief updating. The equality node for s_t is given by

$$f_{=}(s_t, s'_t, s''_t) = \delta(s_t - s'_t)\delta(s_t - s''_t). \quad (2)$$

This equality node ensures that the posterior beliefs over s_t , s'_t , and s''_t are identical.

Finally, the nodes g and h represent the state transition and observation models $p(s_t | s_{t-1}, u_t)$ and $p(y_t | s_t)$, respectively.

Although FFG edges are inherently undirected, we adopt a fixed direction convention, which we later use to indicate the direction of message propagation in the graph.

3.2 Inference by Message Passing

We use a message passing algorithm to perform inference on an FFG. Messages propagate along edges and are updated locally in connected nodes. In this work, we employ the sum-product update rule [21]. Since messages can propagate in both directions along an edge, we refer to messages traveling in the assigned

edge direction as *forward messages*, denoted by $\vec{\mu}$, and messages traveling in the opposite direction as *backward messages*, denoted by $\overleftarrow{\mu}$.

In sum-product message passing, for a general factor node $f(y, x_1, \dots, x_n)$, with incoming messages $\vec{\mu}_{x_i}(x_i)$, the outgoing message toward y is given by [10, 24]

$$\underbrace{\vec{\mu}_y(y)}_{\text{outgoing message}} = \int \underbrace{\vec{\mu}_{x_1}(x_1) \dots \vec{\mu}_{x_n}(x_n)}_{\text{incoming messages}} \underbrace{f(y, x_1, \dots, x_n)}_{\text{node function}} dx_1 \dots dx_n. \quad (3)$$

This rule yields exact Bayesian inference results when the underlying graph is a tree.

When two messages collide at an edge, the posterior marginal belief is obtained by multiplying the messages. More precisely, if the colliding messages are $\vec{\mu}_y(y)$ and $\overleftarrow{\mu}_y(y)$, the marginal belief over y is given by

$$p(y) = \vec{\mu}_y(y) \overleftarrow{\mu}_y(y). \quad (4)$$

For a more detailed explanation of FFGs and BP-based inference, we recommend [21].

3.3 Control as Inference

Control-as-inference reformulates optimal control as a probabilistic inference problem, in which actions are obtained by inferring trajectories that are consistent with both the system dynamics and the desired outcomes [25]. This perspective enables the use of probabilistic graphical models and message-passing algorithms for planning and control.

Active inference, a corollary of the Free Energy Principle [3], is a form of control-as-inference that originated in computational neuroscience as a theory of brain function. In this paper, we use the active inference model proposed in [11]. At each time step t , this model plans T time steps into the future. In the context of the Mountain Car problem, this enables the agent to learn that it must explore the environment and select an appropriate sequence of actions to reach the goal. The model extends the state space model defined in (1) to future time steps by introducing control priors $p(u_k)$ and *preferred* future observations, encoded by $p'(y_k)$. The corresponding extended generative model at the start of time step t can be written as

$$p(y, s, u) \propto \underbrace{p(s_{t-1})}_{\text{state prior}} \prod_{k=t}^{t+T} \underbrace{p(y_k | s_k)}_{\text{observations}} \underbrace{p(s_k | s_{k-1}, u_k)}_{\text{state transition}} \underbrace{p(u_k)}_{\text{control prior}} \underbrace{p'(y_k)}_{\text{preferred observations}}, \quad (5)$$

where $p(u_k)$ represents prior beliefs over a control (action) space, and $p'(y_k)$ encodes prior preferences ("goals") for future observations [12]. The node $p(s_{t-1})$ represents updated beliefs $p(s_{t-1} | y_{1:t-1})$ after having observed $y_{1:t-1}$. The FFG of this model is represented in Figure 3.

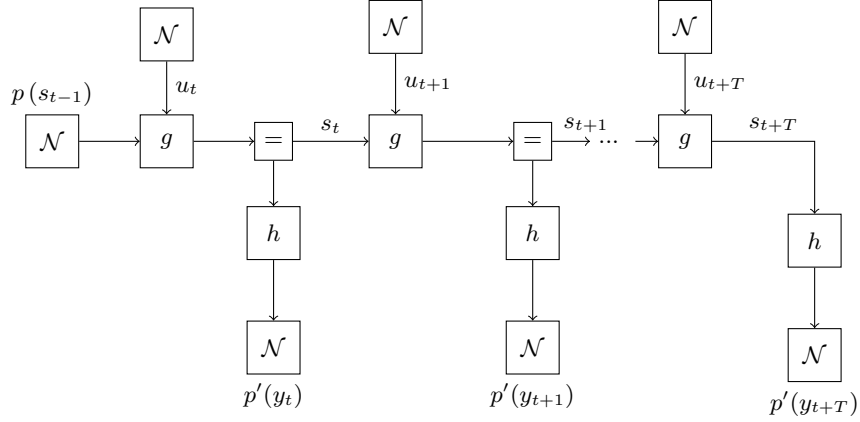


Fig. 3: FFG of the extended model in (5). This model extends the state update model over a horizon of T future time steps from time step t . The "control as inference" objective is to infer the posterior belief for $u_{t:t+T}$ conditioned on preferences $p'(y_{t:t+T})$.

The active inference approach to control does not require an explicit reward function. Instead, rewarding future states (or future outputs, since unobserved future outputs are also latent states) is encoded as an extension of the generative model; consequently, regular inference on this extended model will yield actions that fulfill the preferred future states.

3.4 The Leaky Integrate-and-Fire Neuron Model

This study used the Leaky Integrate-and-Fire (LIF) model [4]. The LIF model simplifies the complex behavior of biological neurons while retaining the essential aspects. The model is based on the membrane potential $V(t)$ at time t , whose dynamics can be described by

$$\tau_{rc} \frac{dV(t)}{dt} = -(V(t) - V_r) + I(t), \quad (6)$$

where $I(t)$ is the input current, V_r is a resting potential, and τ_{rc} is the time constant. Note that, if there is no input current, (6) describes how the membrane potential $V(t)$ decays towards its resting value V_r with a characteristic time constant τ_{rc} [4]. $I(t)$ implies the total stimulation that the neuron received, which can be included as an additional external input or stimulation from another connected neuron group. When postsynaptic currents are applied to a neuron, the membrane potential increases over time and, upon reaching a fixed threshold ϑ , the neuron emits a spike, and the membrane potential resets to its resting value.

3.5 Spike Encoding and Decoding

To implement Belief Propagation in a spiking neural network, continuous probabilistic quantities must be represented by neural activity. In this work, these quantities correspond to Gaussian belief parameters, such as means and variances, that are propagated during message passing. Since spiking neurons communicate through discrete spikes rather than continuous-valued variables, a mechanism is required to encode real-valued means and variances into neural spike trains and decode them into mean and variance estimates.

In computational neuroscience, spike coding schemes are commonly divided into rate coding and temporal coding. Rate coding represents information through the average firing rate of neurons, whereas temporal coding conveys information through the precise timing of spikes [4].

In this work, we employ population coding, a form of rate coding in which information is represented by the joint activity of a neural population. This approach captures important characteristics of biological neural coding while remaining mathematically tractable for probabilistic modeling and analysis [26–29]. Population coding is also supported by biological evidence, including place-cell representations in the hippocampus and neural activity patterns observed in cortical areas of primates and humans [30–33].

In population coding, different neurons become selective to different regions or directions of the represented space, and the represented variable is encoded by the collective activity of many neurons. Such distributed representations are robust to noise and enable accurate representation of continuous-valued variables.

Consider a neural population P consisting of N neurons that represents a vector-valued variable $x \in \mathbb{R}^d$. In our setting, x may denote a belief parameter, for example, the mean of a Gaussian message. Each neuron is assigned an encoder vector $e_i \in \mathbb{R}^d$, which defines the preferred direction of that neuron in the represented space. The preferred direction corresponds to the orientation to which the neuron is most responsive, producing its highest firing activity when the represented variable is aligned with that direction.

We use the encoding and decoding method of the Neural Engineering Framework (NEF) [34]. In this representation, variables (such as mean and variance) are encoded in the neurons’ input currents, which then drive the neuron dynamics given by (6). This current-based representation is required because the LIF neuron model operates on input currents rather than directly on abstract variables. The input current to neuron i is given by

$$I_i(x) = \alpha_i e_i^\top x + I_i^{\text{bias}}, \quad (7)$$

where α_i is the gain and I_i^{bias} is a constant bias current. The term $e_i^\top x$ measures the alignment between the represented variable and the neuron’s preferred direction. The gain α_i scales this projection and controls the sensitivity of the neuron, while I_i^{bias} shifts the baseline current and therefore determines the neuron’s firing threshold. The encoder vectors are randomly sampled and normalized such that

$$\|e_i\|_2 = 1. \quad (8)$$

This creates a diverse set of neuronal tuning curves, allowing the population to represent continuous-valued variables in a distributed manner.

After encoding, the value of the variable x is represented by a distribution of spike activity across the neuronal population. To recover the represented value for downstream computation, neural activity is decoded by

$$\hat{x} = D_P^\top R_P, \quad (9)$$

where $R_P \in \mathbb{R}^N$ denotes the population activity and $D_P \in \mathbb{R}^{N \times d}$ is the decoder matrix for population P . The decoder matrix is obtained by minimizing the reconstruction error over a set of sampled input points:

$$D_P = \arg \min_D \sum_x \|x - D^\top R_P(x)\|^2. \quad (10)$$

This least-squares optimization ensures that the population activity accurately approximates the represented variable over the desired domain. The distributed nature of the decoder computation allows accurate representation despite nonlinear neuronal tuning and redundancy within the neural population [34].

3.6 Networks of Spiking Neurons

We implement transformations using networks of LIF neurons by computing functions through optimized synaptic connection weights. To achieve this, we follow the principles of the NEF [34].

Consider two neural populations, P and Q , with N_P and N_Q neurons, respectively. Population P represents a variable $x \in \mathbb{R}^d$, and the goal is to have population Q represent a transformed variable $z = f(x) \in \mathbb{R}^m$, where $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ is a function.

To implement the transformation, we compute function-specific decoding vectors $D_P^f \in \mathbb{R}^{N_P \times m}$ for population P by solving the least-squares problem

$$D_P^f = \arg \min_D \sum_x \|f(x) - D^\top R_P(x)\|^2, \quad (11)$$

where $R_P(x) \in \mathbb{R}^{N_P}$ is the activity vector of population P in response to input x . The columns $d_i^f \in \mathbb{R}^m$ of D_P^f are the per-neuron decoding vectors, such that $f(x) \approx (D_P^f)^\top R_P(x)$. These decoding vectors are not applied as a separate computational stage in the network; they are used solely to determine the synaptic weights between P and Q , as described next.

The synaptic weight from neuron i in population P to neuron j in population Q is given by

$$\omega_{ij} = \alpha_j e_j^\top d_i^f, \quad (12)$$

where α_j is the gain of neuron j in population Q and $e_j \in \mathbb{R}^m$ is its encoder vector. Each weight ω_{ij} therefore folds the decoding from population P and the encoding into population Q into a single scalar; there is no separate decoder or encoder block between the two populations.

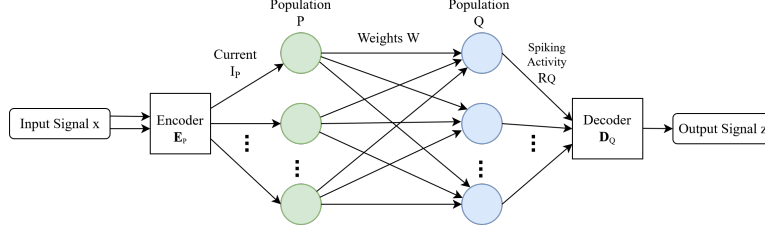


Fig. 4: NEF-based function transformation between two spiking neural populations. Input signal x is encoded into input currents I_P for ensemble P . Synaptic weights W connect population P to population Q , implementing the transformation $f(x)$ implicitly. The decoder D_Q reconstructs the output signal $z \approx f(x)$ from the spiking activity R_Q .

In practice, each neuron in population P emits a spike train, and the postsynaptic input to neuron j in population Q is formed by filtering these spike trains through a synaptic impulse response $k(t)$. Let $a_i(t) = \sum_n k(t - t_n^{(i)})$ denote the filtered activity of neuron i , where $\{t_n^{(i)}\}$ are its spike times and $k(t)$ is an exponential kernel with synaptic time constant τ_s . The resulting input current to neuron j in population Q is

$$I_j(t) = \sum_{i=1}^{N_P} \omega_{ij} a_i(t) + I_j^{\text{bias}} = \alpha_j e_j^\top \left(\sum_{i=1}^{N_P} d_i^f \hat{a}_i(t) \right) + I_j^{\text{bias}}. \quad (13)$$

Since $\sum_i d_i^f \hat{a}_i(t) \approx f(x(t))$, the bracketed term approximates the decoded function value, and (13) reduces to

$$I_j(t) \approx \alpha_j e_j^\top f(x(t)) + I_j^{\text{bias}}, \quad (14)$$

which has exactly the form of (7) with x replaced by $f(x)$. Figure 4 illustrates this two-population architecture. Consequently, population Q encodes the transformed variable $f(x)$ according to the same NEF scheme described in Section 3.5.

This construction allows arbitrary smooth functions to be approximated between neural populations purely through the synaptic weight matrix W , with no explicit intermediate computation. The decoders D_P^f are computed offline during network construction and combined with the encoders of population Q to set the weights; once the network is running, only spikes and synaptic currents are exchanged [34, 35].

4 Methods

In this section, we present the model specification, NEF realization, and inference method for spike-based belief propagation on a factor graph for the Mountain Car problem.

4.1 A State Space Model for Mountain Car Dynamics

We present a realization of the extended generative model (5) for the Mountain Car agent.

First, we describe the state transition model $p(s_t | s_{t-1}, u_t)$. The Mountain Car's state is $s_t = [x_t, \dot{x}_t]^\top$, where x_t is the car's position and $\dot{x}_t$ its velocity. Following [19], the discrete-time transition dynamics are given by

$$\dot{x}_t = \dot{x}_{t-1} + F_g(x_{t-1}) + F_f(\dot{x}_{t-1}) + F_a(u_t) \quad (15a)$$

$$x_t = x_{t-1} + \dot{x}_t, \quad (15b)$$

where F_g , F_f , and F_a are the gravitational, friction, and engine force components. The engine force is a bounded function of the control action

$$F_a(u_t) = F_{\text{lim}} \tanh(u_t). \quad (16)$$

The friction force is linear in velocity,

$$F_f(\dot{x}) = -0.1 \dot{x}, \quad (17)$$

and

$$F_g(x) = \begin{cases} -0.05(2x + 1), & \text{if } x < 0, \\ -0.05[(1 + 5x^2)^{-1/2} + x^2(1 + 5x^2)^{-3/2} + \frac{1}{16}x^4], & \text{otherwise.} \end{cases} \quad (18)$$

determines the horizontal gravitational force of the hilly landscape [11].

The full state transition model is given by

$$p(s_k | s_{k-1}, u_k) = \mathcal{N}(s_k | \tilde{g}(s_{k-1}, u_k), 10^{-4}I), \quad (19)$$

where $\tilde{g}$ is the spike-based realization of

$$g(s_{t-1}, u_t) = \begin{bmatrix} x_{t-1} + \dot{x}_{t-1} + F_g(x_{t-1}) + F_f(\dot{x}_{t-1}) + F_a(u_t) \\ \dot{x}_{t-1} + F_g(x_{t-1}) + F_f(\dot{x}_{t-1}) + F_a(u_t) \end{bmatrix}. \quad (20)$$

Equation 20 follows from (15)–(18).

We assume we observe noisy versions of the states. The observation model of the Mountain Car agent is then given by

$$p(y_k | s_k) = \mathcal{N}(y_k | s_k, 10^{-4}I). \quad (21)$$

We also need to specify the preference prior for future observations. We define the prior $p'(y_k)$ for preferred future observations as

$$p'(y_k) = \begin{cases} \mathcal{N}(y_k | y^*, 10^8 I), & \text{if } k < T - t, \\ \mathcal{N}(y_k | y^*, 10^{-8} I), & T - t \leq k \leq T. \end{cases} \quad (22)$$

Here, y^* denotes the preferred future position state, that is, the target parking location. For a given time step t , the index $k \in \{1, \dots, T\}$ denotes a future step

within the planning horizon. The preference prior is assigned a very large covariance for $k < T - t$, resulting in a broad distribution that permits exploration. The remaining observations are assigned a very small covariance, producing a sharp distribution centered at y^* . As time progresses, the number of the broad prior decreases, while an increasing portion of the horizon becomes strongly constrained toward the target state. Consequently, the agent is allowed to explore early in the trajectory but is progressively driven toward the target observation as the end of the planning horizon approaches. For more details on this approach, we refer the reader to [11].

Finally, to simulate inference in (5), we need to specify the priors

$$p(s_0) = \mathcal{N}(s_0 \mid (-0.5, 0), 10^{-12}I) \quad (23a)$$

$$p(u_k) = \mathcal{N}(u_k \mid 0, 10^{12}) . \quad (23b)$$

4.2 NEF Realization of Mountain Car Dynamics

To implement the Mountain Car dynamics in the NEF framework, we use the construction described in Section 3.6. Separate neural populations are used to approximate F_g , F_f , and F_a , with respectively N_g , N_f , and N_a neurons. Because F_g is approximately linear for $x < 0$ and nonlinear for $x \geq 0$, it is implemented using two separate populations of size N_g , whose outputs are selected according to the sign of x .

For each population, the encoder matrix, gain parameters α_i , bias currents I_i^{bias} , and decoder matrix are defined according to the NEF formulation. In the current work, these parameters are generated automatically using the **Nengo** toolbox [35], which implements the Neural Engineering Framework and initializes heterogeneous neuron properties over the represented input space. Specifically, **Nengo** samples neuron intercepts and maximum firing rates and computes the corresponding gain and bias parameters according to the NEF formulation. As a result, gain values vary across neurons, yielding heterogeneous tuning curves within the population.

The encoder determines the preferred input direction of each neuron, while α and I_{bias} control the neuron firing threshold and response strength. This diversity enables the neural populations to accurately represent the nonlinear functions underlying the transition dynamics.

We also used **Nengo** to automate the optimization tasks in (10) for computing the decoder matrices and connection weights for each network. All computations are performed offline, and the resulting fixed weights and matrices are later used in the spike-based node function, which is integrated into the complete factor graph framework, as described in the following section.

4.3 Nonlinear Message Approximation

Nonlinear factor nodes generally transform Gaussian messages into non-Gaussian distributions. Since our BP framework represents messages as Gaussians, these

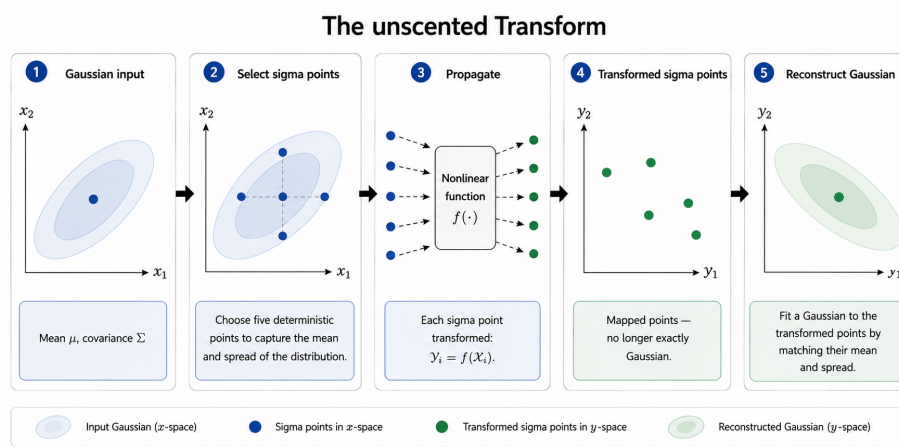


Fig. 5: Illustration of the unscented transform. A Gaussian distribution is represented by a deterministic set of sigma points, which are propagated through a nonlinear function. The transformed points are then used to reconstruct a Gaussian approximation of the resulting distribution.

distributions must be approximated to maintain tractable inference. To this end, we use the Unscented Kalman Filter [36] as a local message-approximation method. As illustrated in Fig. 5, this transformation first represents a Gaussian distribution using a deterministic set of sigma points. These points are propagated through the nonlinear transformation, and the transformed points are then used to reconstruct a Gaussian approximation of the resulting distribution. This allows nonlinear message updates to be incorporated into the Gaussian BP framework without explicit linearization. In our implementation, these UKF-based message updates are automated using the **RxInfer** framework [14].

4.4 Inference Procedure

The overall inference and control procedure is summarized in Algorithm 1. We follow the message passing procedure for active inference agents as detailed in [11]. At each time step, the agent executes the current control action and receives a new observation from the environment. The observation is processed by the factor graph through belief propagation, yielding an updated posterior belief over the latent states. The inferred states are then used to update the posterior distribution over future control actions by performing message passing on the extended planning model. The first action of the resulting control sequence is executed, and the procedure is repeated at the next time step.

Algorithm 1 Inference in the Mountain Car Agent

Require: generative model from section 4.1 $t \leftarrow 0$ **while** $t \leq T$ **do** execute action $u_t \sim p(u_t|y_{1:t-1})$ observe y_t infer state update $p(s_t|y_{1:t})$ (message passing in model, Fig. 2) infer control update $p(u_{t+1:T}|y_{1:t})$ (message passing in ext. model, Fig. 3) $t \leftarrow t + 1$ **end while**

5 Experimental Validation

We evaluated whether the proposed spike-based realization of the nonlinear state-transition factor can be integrated into a belief propagation framework without degrading the control behavior of the original active inference agent. As a reference, we compare our implementation against the active inference controller of Van de Laar and de Vries [11] (the "reference" implementation), which performs inference using conventional numerical computations rather than spike-based neural dynamics.

We used **Nengo** [35] to implement the spike-based realization of the nonlinear functions and to automate the encoding and decoding procedures between continuous-valued variables and neural population activity. **Nengo** was also used to compute the decoder matrices and synaptic connection weights according to (10) and (12). The message-passing procedure and UKF-based message approximations were implemented using the **RxInfer** framework [14], which provides an event-driven realization of belief propagation.

The parameters used in the experiment were $u_0 = 0$, $y^* = 0.4$, $N_g = 300$, $N_f = 100$, $N_a = 100$, and $F_{\text{lim}} = 0.04$.

Figure 6 compares the behavior of the reference active inference controller and the proposed spike-based implementation. The upper plots show the vehicle's position as a function of time, while the lower plots show the corresponding engine control actions. The purpose of this comparison is not to exactly reproduce the control trajectory of the reference implementation, but to verify that the spike-based realization preserves the qualitative control behavior. As shown in Fig. 6, both controllers initially move away from the goal to accumulate momentum and subsequently drive the vehicle toward the target position. In both cases, the vehicle reaches and overshoots the goal position, demonstrating that the proposed spike-based implementation successfully reproduces the intended active inference control strategy.

6 Conclusions and Future Work

This paper presented a spike-based message-passing framework for Bayesian control based on factor graphs and spiking neural networks. By combining belief

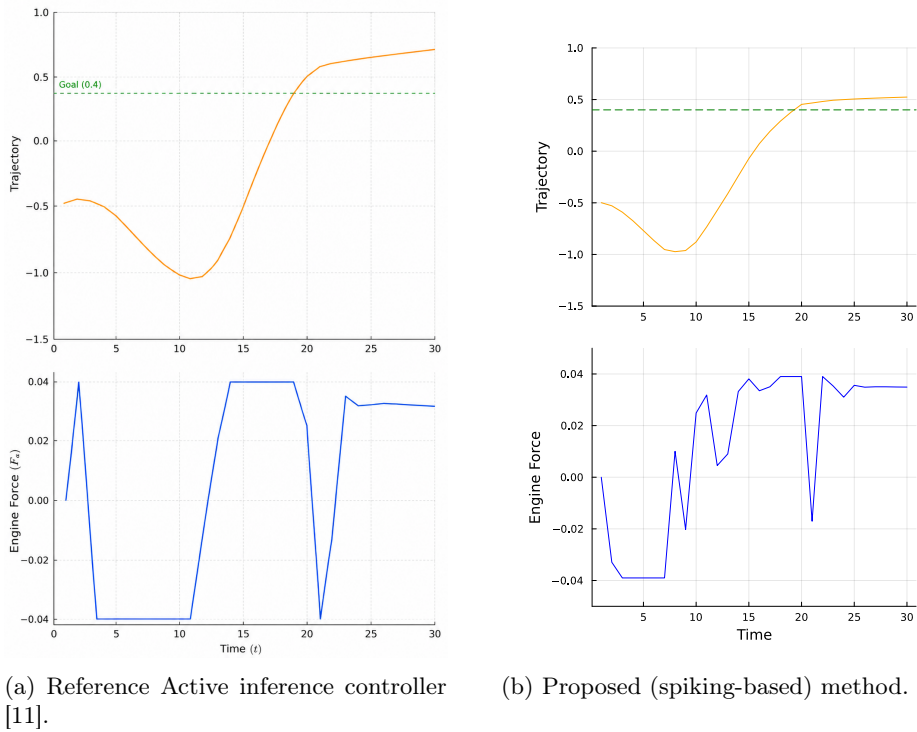


Fig. 6: Comparison of the reference active inference controller (left), versus the proposed controller (right) for the Mountain Car environment. The top panel shows the vehicle position over time, with the dashed horizontal line indicating the goal position. The bottom panel shows the corresponding engine force applied at each time step. Both methods exhibit the characteristic Mountain Car strategy of initially moving away from the goal to build momentum before ascending the hill. In both cases, the vehicle successfully reaches and overshoots the goal position, demonstrating successful task completion.

propagation with SNNs, we developed a biology-inspired implementation of a nonlinear, multivariate state transition factor for a Mountain Car agent. The proposed approach integrates spike-based computation with probabilistic inference, enabling state estimation and control through local message passing on a factor graph.

The resulting model was implemented using **Nengo** and integrated into an event-driven inference framework using **RxInfer**. Experimental results demonstrated that the proposed controller successfully solves the Mountain Car problem. These results indicate that spike-based neural dynamics can support nonlinear Bayesian inference and control in a biologically inspired manner.

Interesting directions for future work include extending the framework to a fully spike-based implementation, in which both the factor nodes and the

message-passing algorithm are realized through spiking neural dynamics. Such an implementation would enable deployment on neuromorphic hardware and facilitate the evaluation of its energy efficiency.

Another promising direction is to replace the hand-crafted state-transition model with a learned dynamics model. Rather than relying on predefined equations, the state-transition factor could be learned directly from interactions with the environment, allowing the framework to be applied to more complex and less structured control problems.

Acknowledgments. We gratefully acknowledge funding for this study by the Eindhoven Artificial Intelligence Systems Institute (EAISI) at Eindhoven University of Technology.

References

1. Yuguo Yu, Peter Herman, Douglas L Rothman, Divyansh Agarwal, and Fahmeed Hyder. Evaluating the gray and white matter energy budgets of human brain function. *Journal of Cerebral Blood Flow & Metabolism*, 38(8):1339–1353, 2018.
2. Kenji Doya. *Bayesian brain: Probabilistic approaches to neural coding*. MIT Press, 2007.
3. Karl Friston, James Kilner, and Lee Harrison. A free energy principle for the brain. *Journal of Physiology*, 100(1-3):70–87, 2006.
4. Wulfram Gerstner and Werner M Kistler. *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, Cambridge, UK, 2002.
5. Kaushik Roy, Akhilesh Jaiswal, and Priyadarshini Panda. Towards spike-based machine intelligence with neuromorphic computing. *Nature*, 575(7784):607–617, 2019.
6. Hadi Vafaii, Dekel Galor, and Jacob L Yates. Brain-like variational inference. *arXiv preprint arXiv:2410.19315*, 2024.
7. Andreas Steimer, Wolfgang Maass, and Rodney Douglas. Belief propagation in networks of spiking neurons. *Neural Computation*, 21(9):2502–2523, 2009.
8. Vincent Bouttier. *Circular belief propagation as a model for optimal and suboptimal inference in the brain: extending the algorithm and proposing a neural implementation*. PhD thesis, Université Paris Cité, 2021.
9. Sophie Deneve. Bayesian inference in spiking neurons. *Advances in Neural Information Processing Systems*, 17:353–360, 2004.
10. Frank R Kschischang, Brendan J Frey, and Hans-Andrea Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2002.
11. Thijs W van de Laar and Bert de Vries. Simulating active inference processes by message passing. *Frontiers in Robotics and Artificial Intelligence*, 6(20):1–15, 2019.
12. Sepideh Adamiat, Wouter M Kouw, Bart van Erp, and A Bert de Vries. Message passing-based Bayesian control of a cart-pole system. In *International Workshop on Active Inference*, Communications in Computer and Information Science, pages 209–221, 2024.
13. Bart van Erp, Dmitry Bagaev, Albert Podusenko, İsmail Şenöz, and Bert de Vries. Multi-agent trajectory planning with NUV priors. In *IEEE American Control Conference*, pages 2766–2771, 2024.

14. Dmitry Bagaev and Bert de Vries. Reactive message passing for scalable Bayesian inference. *Scientific Programming*, 2023(6601690):1–26, 2023.
15. Thomas Ott and Ruedi Stoop. The neurodynamics of belief propagation on binary markov random fields. *Advances in Neural Information Processing Systems*, 19:1057–1064, 2006.
16. Wolfgang Maass. Liquid state machines: Motivation, theory, and applications. In *Computability in Context: Computation and Logic in the Real World*, pages 275–296. World Scientific, 2011.
17. Sepideh Adamiyat, Wouter M Kouw, and Bert de Vries. A spiking neural network implementation of Gaussian belief propagation. *Neuromorphic Computing and Engineering*, 2025. <https://doi.org/10.1088/2634-4386/ae6f46>.
18. Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*, volume 1. MIT Press, 1998.
19. Kai Ueltzhöffer. Deep active inference. *Biological Cybernetics*, 112(6):547–573, 2018.
20. Karl Friston. Life as we know it. *Journal of the Royal Society Interface*, 10(86), 2013.
21. Hans-Andrea Loeliger, Justin Dauwels, Junli Hu, Sascha Korl, Li Ping, and Frank R Kschischang. The factor graph approach to model-based signal processing. *Proceedings of the IEEE*, 95(6):1295–1322, 2007.
22. Karl J Friston, Thomas Parr, and Bert de Vries. The graphical brain: Belief propagation and active inference. *Network Neuroscience*, 1(4):381–414, 2017.
23. G David Forney. Codes on graphs: Normal realizations. *IEEE Transactions on Information Theory*, 47(2):520–548, 2001.
24. Hans-Andrea Loeliger. An introduction to factor graphs. *IEEE Signal Processing Magazine*, 21(1):28–41, 2004.
25. Hilbert J Kappen, Vicenç Gómez, and Manfred Opper. Optimal control as a graphical model inference problem. *Machine Learning*, 87(2):159–182, 2012.
26. Si Wu, Shun-ichi Amari, and Hiroyuki Nakahara. Population coding and decoding in a neural field: a computational study. *Neural Computation*, 14(5):999–1026, 2002.
27. Richard S Zemel, Peter Dayan, and Alexandre Pouget. Probabilistic interpretation of population codes. *Neural Computation*, 10(2):403–430, 1998.
28. Wei Ji Ma, Jeffrey M Beck, Peter E Latham, and Alexandre Pouget. Bayesian inference with probabilistic population codes. *Nature Neuroscience*, 9(11):1432–1438, 2006.
29. Terence David Sanger. Probability density estimation for the interpretation of neural population codes. *Journal of Neurophysiology*, 76(4):2790–2793, 1996.
30. John O’Keefe and Jonathan Dostrovsky. The hippocampus as a spatial map: preliminary evidence from unit activity in the freely-moving rat. *Brain Research*, 34:171–175, 1971.
31. Francesco Ceccarelli, Lorenzo Ferrucci, Fabrizio Londei, Surabhi Ramawat, Emiliano Brunamonti, and Aldo Genovesio. Static and dynamic coding in distinct cell types during associative learning in the prefrontal cortex. *Nature Communications*, 14(8325):1–17, 2023.
32. John H Maunsell and David C Van Essen. Functional properties of neurons in middle temporal visual area of the macaque monkey. I. selectivity for stimulus direction, speed, and orientation. *Journal of Neurophysiology*, 49(5):1127–1147, 1983.

33. Wan-Yu Shih, Hsiang-Yu Yu, Cheng-Chia Lee, Chien-Chen Chou, Chien Chen, Paul W Glimcher, and Shih-Wei Wu. Electrophysiological population dynamics reveal context dependencies during decision making in human frontal cortex. *Nature Communications*, 14(1):7821, 2023.
34. Chris Eliasmith and Charles H Anderson. *Neural engineering: Computation, representation, and dynamics in neurobiological systems*. MIT Press, 2003.
35. Trevor Bekolay, James Bergstra, Eric Hunsberger, Travis DeWolf, Terrence C Stewart, Daniel Rasmussen, Xuan Choo, Aaron Russell Voelker, and Chris Eliasmith. Nengo: a python tool for building large-scale functional brain models. Technical report, 2014.
36. Simon J Julier and Jeffrey K Uhlmann. New extension of the Kalman filter to nonlinear systems. In *Signal Processing, Sensor Fusion, and Target Recognition VI*, volume 3068, pages 182–193. SPIE, 1997.